Hello World of MCP Servers

Estimated time needed: **30** minutes

This lab goes through the quick process of creating a **simple** MCP Server and Client using the FastMCP framework that wraps the official MCP SDK.

Objectives

After completing this lab you will be able to:

- Use FastMCP to create MCP Servers in either **STDIO** and **HTTP** transports
- Register custom tools, resources, and prompts to an MCP Server
- Test MCP Servers with Client connections and manual tool calling
- Create a Multi-server client and ReAct agent to use all our MCP tools

---

Setup

For this lab, we will be using the following libraries:

- `fastmcp` for creating and configuring MCP servers and clients
- `langchain` for tooling convention and usage
- `langchain-mcp-adapters` for adapting MCP tools to be used in LangChain's ecosystem
- `langgraph` for creating agents using LangChain principles

Installing Required Libraries

Run the following command to install the required libraries.

In [1]:
```
%%capture
%pip install fastmcp==2.12.2
%pip install langchain==0.3.27
%pip install langchain_mcp_adapters==0.1.9
%pip install langgraph==0.6.7
%pip install langchain_openai==0.3.33
```

Importing Required Libraries

Let's import necessary dependencies and libraries at the top here.

In [2]:
```
import socket
import asyncio
from fastmcp import FastMCP, Client
```

Create the actual directory path where files will be stored. This is an **actual location** on your file system

In [3]:
```
import os
def make_dir():
    if os.path.exists("path"):
        print("✓ Path directory already exists")
    else:
        print("✗ Path directory doesn't exist - creating it...")
        os.makedirs("path")
        print("✓ Path directory created")
```

In this JupyterLab environment, we have to be careful running HTTP servers. Running a server on a port that is in use or killing a server on a running port causes the Python kernel to crash and you will have to rerun all the code cells. To deal with this, let's make a helper function to check the port we want to run our MCP server on.

Choose a port (default 8000) for the `PORT` variable and run the following cell:

In [4]:
```
PORT = 8000

def test_port(port=PORT):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            s.bind(('127.0.0.1', port))
            return False
        except socket.error:
            return True

f"Port {PORT} is available: {not test_port()}"
```

Out[4]:
```
'Port 8000 is available: True'
```

Print information about the read/write streams and session ID.

In [5]:
```
def print_stream_info(read, write, _sid, verbose=False):
    """Print information about the read/write streams and session ID.
    """
    if verbose:
        print("READ (receives FROM server):")
        print(read)
        print()
```

```
        print("WRITE (sends TO server):")
        print(write)
        print()

        print("SESSION ID:")
        print(_sid())
```

Tools

If you are familiar with agentic principles, you probably know about tools and their functionality. Let's take a quick look at tooling in **LangChain**.

In [6]:
```
from langchain_core.tools import tool
```

This imports the `@tool` decorator, which converts regular Python functions into **LangChain compatible** tools that work locally within a single application, To define a tool we simply write the decorator over a Python function. We can also access the tool information and call it using the `.invoke()` method.

In [7]:
```
@tool
def multiply(a: int, b: int) -> int:
   """Multiply two numbers."""
   return a * b

print(multiply.name)
print(multiply.description)
print(multiply.args)

print("What is 2 x 3?")
print("Answer: " + str(multiply.invoke({"a": 2, "b": 3})))
```

```
multiply
Multiply two numbers.
{'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B', 'type': 'intege
r'}}
What is 2 x 3?
Answer: 6
```

Creating a Calculator MCPServer

MCP servers behave like functions that AI agents can call to perform actions using. Let's define and configure a basic MCP server.

- **FastMCP** creates the server instance with a name and instructions
- `@mcp.tool` decorator registers functions as callable tools
- `@mcp.prompt` decorator creates reusable prompt templates
- `@mcp.resource` decorator exposes data resources with URI patterns

This creates a complete MCP server named **CalculatorMCPServer** with mathematical tools (add, subtract), a document reading resource, and a code review prompt template.

FastMCP Object

The first step is to create an MCP server object `mcp` for the calculator. Like before, the object will be used to control the server. The parameters are:

- `name` : "CalculatorMCPServer" - the name of the server
- `instructions` : "This server provides mathematical calculation tools. Call `add()` and `subtract()` to perform basic arithmetic operations." - this describes the functionality of the server

In [8]:
```
from fastmcp import FastMCP

mcp = FastMCP(
    name="CalculatorMCPServer",
    instructions="""
        This server provides data analysis tools.
        Call get_average() to analyze numerical data.
    """)
print('mcp object',mcp)
```

```
mcp object FastMCP('CalculatorMCPServer')
```

Tools

Tools (Active Capabilities) - Functions that AI agents can call to perform actions - Similar to LangChain tools but networked and discoverable

We define the MCP tools `add` and `subtract` - these will be called on the MCP object.

In [9]:
```
@mcp.tool
def add(a: int, b: int) -> int:
    """
    Add two integers together.

    Args:
        a (int): The first integer.
        b (int): The second integer.

    Returns:
        int: The sum of `a` and `b`.

    Example:
        >>> add(3, 5)
        8
    """
    return a + b


@mcp.tool
def subtract(a: int, b: int) -> int:
    """
```

```
    Subtract one integer from another.

    Args:
        a (int): The number to subtract from.
        b (int): The number to subtract.

    Returns:
        int: The result of `a - b`.

    Example:
        >>> subtract(10, 4)
        6
    """
    return a - b
```

Resources

Resources are like filing cabinets that AI systems can open to read information. Think of them as "files" or "data sources" that AI can easily access using simple web-like addresses for finding exactly what they need.

We create resources using the `@mcp.resource` decorator. When you set up something like `file:///endpoint/{name}`, you're basically creating an address system where **named path parameter** gets replaced with whatever file the AI wants to read.

The URI is purely for resource identification, name the URI path something that fits the type, style, and content of resource you are exposing.

Let's create two resources, one that returns a prewritten message template and another that returns the contents of an actual file.

In [10]:
```
@mcp.resource("file:///endpoint/{name}")
def return_template_document(name: str) -> str:
    """Read a document by name"""
    return f"Document contents of {name}"
```

Create the actual directory path where files will be stored. This is an **actual location** on your file system

In [11]:
```
make_dir()
```

```
✗ Path directory doesn't exist - creating it...
✓ Path directory created
```

In [12]:
```
%%capture
!wget -P path/ https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
!wget -P path/ https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
```

When you call this resources, it will require the input `{name}` to identify which document to retrieve. `path/{name}` is where files actually exist on disk. The function

reads from this physical file system location to return the content.

Note: The URI endpoint is the MCP address for requesting resources, while the path is the actual storage location on your system. They are related but distinct.

In [13]:
```
@mcp.resource("file://endpoint2/{name}")
def read_document(name: str) -> str:
    """Read a document by name from the path directory"""
    try:
        # Read from the actual file system path
        with open(f"path/{name}", "r") as f:
            return f.read()
    except FileNotFoundError:
        return f"Document '{name}' not found in path directory"
    except Exception as e:
        return f"Error reading document: {str(e)}"
```

Prompts

Prompts are consistent, reusable templates that can be called for simple, repetitive tasks. They capture domain expertise in a structured way, so instead of reinventing instructions each time, the AI can rely on a proven pattern.

In [14]:
```
@mcp.prompt(title="Code Review")
def review_code(code: str) -> str:
    return f"Please review this code:\n\n{code}"
```

Creating a Client - In-Memory Transport

Now that we've created our MCP server with tools, let's create a client to test it. For this first example, we'll use the simplest connection method: in-memory transport. When your server and client are running in the same Python process, you can connect directly without creating a separate transport object. In a Pythonic sense, we'll use the client object to call the tools you need via methods. However, because it's actually communicating with a server, you will have to use several special Python functions to communicate with the server, such as async and await for **asynchronous operations**

In [15]:
```
from fastmcp import  Client

client = Client(mcp)
print(f"client: {client}")
```

```
client: <fastmcp.client.client.Client object at 0x77aa1a5d9fd0>
```

Tools

We create a client to test our MCP server and call its tools remotely. Even though the server is running in the same process (in-memory transport), we still follow the MCP

protocol pattern of client-server communication. The `call_tool()` method invokes a specific tool with parameters. When we call `client.call_tool` we are not really calling a local object, we are calling a server through the client

We wrap this in a function because we are dealing with a server that may take time to respond we will always use the following asynchronous functions :

- `async with client` : The server communication involves waiting time, so we use the async context manager which automatically manages the client lifecycle (opening and closing the connection)
- `await` : executes the async function and allows the Python process to wait for the server's response without blocking

In [16]:
```
async def call_add_tool(a: int, b: int):
    async with client:
        result = await client.call_tool("add", {"a": a, "b": b})
        return result
```

We can use the function that wraps the `async` context manager to call the add tool here:

In [17]:
```
response = await call_add_tool(4, 5)
response
```

Out[17]:
```
CallToolResult(content=[TextContent(type='text', text='9', annotations=Non
e, meta=None)], structured_content={'result': 9}, data=9, is_error=False)
```

There are several ways to get the result via the attribute:

In [20]:
```
# The actual answer/data
print("\nResult Data .data :")
print(response.data)  # 9

# Content (text format)
print("\nContent (as text):")
print(response)  # "9"

# Structured content (as dictionary)
print("\nStructured Content:")
print(response.structured_content)
```

```
Result Data .data :
9

Content (as text):
CallToolResult(content=[TextContent(type='text', text='9', annotations=None,
meta=None)], structured_content={'result': 9}, data=9, is_error=False)

Structured Content:
{'result': 9}
```

Question Write a function to call the subtract tool with parameters a=4 and b=5, then execute it and print the result. What will be the output?

▶ Details

Click here for hint

```
async def call_subtract_tool(a: int, b: int):
    async with client:
        result = await client.call_tool("subtract", {"a": a, "b":
b})
        return result

response = await call_subtract_tool(4, 5)
print(response.content[0].text)
</details> ```
```

The client object has useful attributes to get information about the server. Like before, we use asynchronous operations to communicate with the server. We use `await client.list_tools()` to get all available tools. Then we loop through and print each tool's name and description.

In [21]:
```
async with client:
    tools = await client.list_tools()
    print("Available tools:")
    for tool in tools:
        print(f"- {tool.name}: {tool.description}")
```

```
Available tools:
- add: Add two integers together.

Args:
    a (int): The first integer.
    b (int): The second integer.

Returns:
    int: The sum of `a` and `b`.

Example:
    >>> add(3, 5)
    8
- subtract: Subtract one integer from another.

Args:
    a (int): The number to subtract from.
    b (int): The number to subtract.

Returns:
    int: The result of `a - b`.

Example:
    >>> subtract(10, 4)
    6
```

We can obtain the first tool `object` from the list

In [22]:
```
tool_obj = tools[0]
print(tool_obj)
```

```
name='add' title=None description='Add two integers together.\n\nArgs:\n
a (int): The first integer.\n    b (int): The second integer.\n\nReturns:\n
int: The sum of `a` and `b`.\n\nExample:\n    >>> add(3, 5)\n    8' inputSch
ema={'properties': {'a': {'title': 'A', 'type': 'integer'}, 'b': {'title':
'B', 'type': 'integer'}}, 'required': ['a', 'b'], 'type': 'object'} outputSc
hema={'properties': {'result': {'title': 'Result', 'type': 'integer'}}, 'req
uired': ['result'], 'title': '_WrappedResult', 'type': 'object', 'x-fastmcp-
wrap-result': True} icons=None annotations=None meta={'_fastmcp': {'tags':
[]}} execution=None
```

`Input Schema` Defines how the tool takes arguments in a standardized JSON format, necessary because data is transmitted over the internet/network between client and server.

In [23]:
```
input_schema = tool.inputSchema
print(input_schema)
```

```
{'properties': {'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B',
'type': 'integer'}}, 'required': ['a', 'b'], 'type': 'object'}
```

`Output Schema`: JSON structure defining what the tool returns (data type and structure of the response) - necessary because responses are transmitted over the network as JSON; tells clients (and LLMs) what to expect back. MCP doesn't always expose this since some tools perform actions (like saving to database) where the LLM only needs simple confirmation, not detailed output data.

In [24]:
```
output_schema = tool.outputSchema
print(output_schema )
```

```
{'properties': {'result': {'title': 'Result', 'type': 'integer'}}, 'require
d': ['result'], 'title': '_WrappedResult', 'type': 'object', 'x-fastmcp-wrap
-result': True}
```

Resources

We'll call the `read_resource` method in an asynchronous context manager wrapped in a function. The parameter for reading the resource is the URI with our configured `name`.

In [25]:
```
async def call_resource(name):
    async with client:
        result = await client.read_resource(f"file:///endpoint/{name}")
        return result
```

Use the `call_resource` function that wraps the async context manager to call the document contents here, it (`endpoint`) should return the template message.

In [26]:
```
response = await call_resource("README.txt")
print(response[0].text)
```

```
Document contents of README.txt
```

We define the function as follows:

In [27]:
```
async def call_resource2(name):
    async with client:
        result = await client.read_resource(f"file://endpoint2/{name}")
        return result
```

This time when we input the parameter **README.txt**, it will retrieve the actual file from the disk which is referenced by `endpoint2/README.txt`.

In [28]:
```
response = await call_resource2("README.txt")
```

And if a requested file doesn't exist at `path` MCP servers typically handle file not found errors automatically and return error messages:

In [36]:
```
response = await call_resource2("README.txt")
resource = response[0]
print(response)
```

```
[TextResourceContents(uri=AnyUrl('file://endpoint2/README.txt'), mimeType='t
ext/plain', meta=None, text='# Documents Folder\n\nThis folder contains docu
ments accessible through the Calculator MCP Server.\n\n## Available Document
s:\n\n- examples.txt - Examples of how to use the calculator\n- README.txt -
This file\n')]
```

we can print out the attrabutes of the objects

In [33]:
```
print(f"uri:      {resource.uri}")
print(f"mimeType: {resource.mimeType}")
print(f"meta:     {resource.meta}")
print(f"text:     {resource.text}")
print(endpoint2)
```

```
uri:      file://endpoint2/README.txt
mimeType: text/plain
meta:     None
text:     # Documents Folder

This folder contains documents accessible through the Calculator MCP Server.

## Available Documents:

- examples.txt - Examples of how to use the calculator
- README.txt - This file
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[33], line 5
      3 print(f"meta:     {resource.meta}")
      4 print(f"text:     {resource.text}")
----> 5 print(endpoint2)

NameError: name 'endpoint2' is not defined
```

Prompts

We'll also create a function to call the prompt method to review code. The only parameter is the code content to be reviewed.

In [37]:
```
async def call_prompt(code):
    async with client:

        result = await client.get_prompt("review_code", {"code": code})
        return result
```

We can use the function that wraps the async context manager to call the prompt template here:

In [38]:
```
response = await call_prompt("CODE TO BE REVIEWED")
```

The response contains several attributes, but the messages attribute is what we need. It contains a PromptMessage object with two key parts:

`content` : The actual prompt text

`role` : The role the prompt is intended for (e.g., 'user', 'assistant')

In [39]:
```
message=response.messages[0]
print(f"Prompt Role:{message.role}")
print(f"Prompt Content:{message.content.text}")
```

```
Prompt Role:user
Prompt Content:Please review this code:

CODE TO BE REVIEWED
```

HTTP Transport MCP Servers

HTTP transport allows MCP servers to run as web services that clients connect to via URLs. This is ideal for remote servers, cloud deployments, or when you want multiple clients to share the same server instance.

First, check if the port is available (True). If available, run the following cell to start the MCP server on that port. If the port is unavailable, change the PORT variable and rerun the server. This requirement is just for JupyterLab.

In [40]:
```
f"Port {PORT} is available: {not test_port()}"
```

Out[40]:
```
'Port 8000 is available: True'
```

Starting HTTP MCP Server

Let's launch the MCP server as an HTTP service running in the background. Using the `mcp` object, we call the method `mcp.run_http_async()` which starts the HTTP server on the specified port:

```
mcp.run_http_async(port=PORT)
```

- Server runs continuously in the background
- `/mcp` endpoint is automatically created for JSON-RPC communication

for jupyter notebook only However, because we are running this in a Jupyter notebook, we have to use `asyncio.create_task()`, which runs the server concurrently without blocking, allowing us to continue using other cells

In [41]:
```
#server_task = asyncio.create_task(server_task_)
asyncio.create_task(mcp.run_http_async(port=PORT))
print(f"HTTP MCP Server started in background on port {PORT}")
```

```
HTTP MCP Server started in background on port 8000
```

```
╭────────────────────────────────────────────────────────────────────────────╮
│                                                                            │
│                                                                            │
│        _ __ ___  _____           __  __  ____________    ____    ____      │
│       _ __ ___ .'____/___ _____/ /_/  |/  / ____/ __ \  |___ \  / __ \     │
│      _ __ ___ / /_  / __ `/ ___/ __/ /|_/ / /   / /_/ /  ___/ / / / / /    │
│     _ __ ___ / __/ / /_/ (__  ) /_/ /  / / /___/ ____/  /  __/_/ /_/ /     │
│    _ __ ___ /_/    \____/____/\__/_/  /_/\____/_/      /____(*)____/       │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                                FastMCP  2.0                                │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
│                 🖥️  Server name:     CalculatorMCPServer                   │
│                                                                            │
│                 📦  Transport:       Streamable-HTTP                       │
│                                                                            │
│                 🔗  Server URL:      http://127.0.0.1:8000/mcp             │
│                                                                            │
│                                                                            │
│                                                                            │
│                 🏎️  FastMCP version: 2.12.2                                │
│                                                                            │
│                 🤝  MCP SDK version: 1.26.0                                │
│                                                                            │
│                                                                            │
│                                                                            │
│                 📚  Docs:            https://gofastmcp.com                 │
│                                                                            │
│                 🚀  Deploy:          https://fastmcp.cloud                 │
│                                                                            │
│                                                                            │
│                                                                            │
│                                                                            │
╰────────────────────────────────────────────────────────────────────────────╯
```

```
[03/16/26 04:20:09] INFO     Starting MCP server 'CalculatorMCPServer'
                             http://127.0.0.1:8000/mcp
```

```
INFO:     Started server process [299]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

IMPORTANT NOTE

If you want to **change** or **rerun** the server at any point, you **MUST** change the port as to not crash the kernel.

HTTP Transport and Client

HTTP Transport

Now we'll create an MCP client that uses `HTTP` transport to communicate with the server. This creates a transport object that tells the MCP Client how to communicate with the MCP server over HTTP.

In [42]:
```
from fastmcp.client.transports import StdioTransport, StreamableHttpTranspor
transport_http = StreamableHttpTransport(
    url=f"http://127.0.0.1:{PORT}/mcp")
```

Now let's create the client using `transport_http` object

In [43]:
```
http_client = Client(transport_http)
print('http_client',http_client )
```

```
http_client <fastmcp.client.client.Client object at 0x77aa19576060>
```

We define an async function `test_client_http` that takes a client object and two integers as parameters. The function uses an async context manager (`async with client`) to establish a connection to the MCP server, then calls the "add" tool with the provided arguments and returns the result, .

In [44]:
```
async def test_client_http(client: Client, a: int, b: int)->int:
    async with client:
        result = await client.call_tool("add", {"a": a, "b": b})
        return result
```

We call the `test_client` function with our HTTP client and the values 4 and 5, awaiting the result. The response contains the tool's output, which we access by navigating to the first content item's text property and printing it.

In [45]:
```
response = await test_client_http(http_client, 4, 5)
print(response.content[0].text)
```

```
INFO:     127.0.0.1:49968 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:49984 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:49998 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50010 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50018 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:50022 - "DELETE /mcp HTTP/1.1" 200 OK
9
```

```
/opt/conda/lib/python3.12/contextlib.py:105: DeprecationWarning: Use `stream
able_http_client` instead.
  self.gen = func(*args, **kwds)
```

Exercise

Write an async function called get_tool_list that takes a Client object as a parameter and returns a list of all available tools from the MCP server. Then call this function with the http_client and print the results.

In [56]:
```
async def get_tool_list(client: Client):
    async with client:
        abstools = await client.list_tools()
        return  abstools

tool_list=await get_tool_list(http_client)
print(tool_list[0].description)
```

```
INFO:     127.0.0.1:33752 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33754 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:33766 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33768 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:33770 - "DELETE /mcp HTTP/1.1" 200 OK
Add two integers together.

Args:
    a (int): The first integer.
    b (int): The second integer.

Returns:
    int: The sum of `a` and `b`.

Example:
    >>> add(3, 5)
    8
```

```
/opt/conda/lib/python3.12/contextlib.py:105: DeprecationWarning: Use `stream
able_http_client` instead.
  self.gen = func(*args, **kwds)
```

▶ Click here for Solution

LangChain Tools with HTTP MCP Servers

Converting an MCP tool to a LangChain tool is relatively simple. The main challenge is that it requires a more complex client and transport layer — langchain-mcp-adapters. The `ClientSession` essentially acts as a client that retrieves information from `StreamableHttpTransport`, which is a bit more complex. Before we dive deeper into `StreamableHttpTransport`, let's import `ClientSession`, `load_mcp_tools`, `create_react_agent`, and our LLM.

In [57]:
```
from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI
from mcp import ClientSession
llm = "openai:gpt-5-nano"
```

The `streamablehttp_client` function connects you to an MCP server over HTTP. When you call it, it opens a connection and gives you back three things that let you communicate with the server:

`read:` Receives messages from the server

`write:` Sends messages to the server

`_sid:` Returns your unique connection ID

In [58]:
```
from mcp.client.streamable_http import streamablehttp_client
async with streamablehttp_client(f"http://127.0.0.1:{PORT}/mcp") as (read, w
    print_stream_info(read, write, _sid, verbose=True)
```

```
READ (receives FROM server):
MemoryObjectReceiveStream(_state=MemoryObjectStreamState(max_buffer_size=0,
buffer=deque([]), open_send_channels=1, open_receive_channels=1, waiting_rec
eivers=OrderedDict(), waiting_senders=OrderedDict()), _closed=False)

WRITE (sends TO server):
MemoryObjectSendStream(_state=MemoryObjectStreamState(max_buffer_size=0, buf
fer=deque([]), open_send_channels=1, open_receive_channels=1, waiting_receiv
ers=OrderedDict(), waiting_senders=OrderedDict()), _closed=False)

SESSION ID:
None
```

```
/opt/conda/lib/python3.12/contextlib.py:105: DeprecationWarning: Use `stream
able_http_client` instead.
  self.gen = func(*args, **kwds)
```

To use MCP tools with a LangChain agent, we first establish a connection and initialize the session. Then load_mcp_tools converts the MCP tools into LangChain-compatible format. We create the agent with our LLM and converted tools, then make a query. Everything stays wrapped in async context managers to keep the session alive while the agent we use `ainvoke` not `invoke` for the agent .

In [59]:
```
async with streamablehttp_client(f"http://127.0.0.1:{PORT}/mcp") as (read, w
    async with ClientSession(read, write) as session:
        await session.initialize()

        # Load LangChain tools from the LIVE MCP session
        tools = await load_mcp_tools(session)

        # Build the agent WHILE the session is still open
        agent = create_react_agent(
            model=llm,
```

```
            tools=tools,
        )
        agent_response = await agent.ainvoke({"messages": "Use the add tool
```

```
INFO:     127.0.0.1:39636 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:39652 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:39658 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:39660 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:39662 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:47246 - "DELETE /mcp HTTP/1.1" 200 OK
```

you can output the tool responce

In [60]:
```
print(agent_response['messages'][-1].content)
```

```
Yes. I used the add tool to compute 2 + 1. The tool returned 3, so the sum i
s 3.
```

STDIO Transport in MCP Servers

For local MCP servers using STDIO transport, the client spawns the server as a **child process** and communicates through the server's `stdin/stdout` pipes. This is different from HTTP, where the client connects to a URL on an already-running web server.

We can't run the server as a regular Python function in Jupyter, so we save it as a separate `.py` file. The code is identical to what we've seen before, except for the critical addition of `mcp.run()` at the end, which initializes the MCP server to listen on stdin/stdout.

```
from fastmcp import FastMCP

mcp = FastMCP(
    name="CalculatorMCPServer",
    instructions="""
        This server provides data analysis tools.
        Call add() to add two numbers.
    """
)

@mcp.tool
def add(a: int, b: int) -> int:
    """Adds two integer numbers together."""
    return a + b

@mcp.tool
def subtract(a: int, b: int) -> int:
    """Subtracts b from a"""
    return a - b

@mcp.resource("file://documents/{name}")
def read_document(name: str) -> str:
```

```
    """Read a document by name"""
    return f"Document contents of {name}"

@mcp.prompt(title="Code Review")
def review_code(code: str) -> str:
    return f"Please review this code: {code}"

if __name__ == "__main__":
    mcp.run()
```

We'll save the Python code as a string variable called `server_content`:

In [61]:
```
server_content = '''
from fastmcp import FastMCP

mcp = FastMCP(
    name="CalculatorMCPServer",  # Fixed: added missing comma
    instructions="""
        This server provides data analysis tools.
        Call add() to add two numbers.
    """
)

@mcp.tool
def add(a: int, b: int) -> int:
    """Adds two integer numbers together."""
    return a + b

@mcp.tool
def subtract(a:int, b:int) -> int:
    """Subtracts b from a"""
    return a - b

@mcp.resource("file://documents/{name}")
def read_document(name: str) -> str:
    """Read a document by name"""
    return "Document contents of {name"

@mcp.prompt(title="Code Review")
def review_code(code: str) -> str:
    return f"Please review this code: {code}"

if __name__ == "__main__":
    mcp.run()
'''
```

then write it to a file named `stdio_server.py`:

In [63]:
```
with open('stdio_server.py', 'w') as f:
    f.write(server_content)

print("Created corrected stdio_server.py file")
```

```
Created corrected stdio_server.py file
```

Configuring STDIO Transport

Now let's create a StdioTransport to communicate over STDIO transport by defining the command and arguments to launch the server. This will start the server using our `stdio_server.py` file.

The `StdioTransport` function specifies the command and arguments to launch the server:

`command="python"` tells the system to use the Python interpreter

`args=["stdio_server.py"]` provides the script to execute

In [64]:
```
transport_stdio = StdioTransport(
    command="python",
    args=["stdio_server.py"]
)
```

note: Relative paths work well in Jupyter when the notebook and server are in the same directory Absolute paths are recommended for production environments for reliability

The above configuration method is common when adding MCP servers. For example, some clients (e.g., Cursor, Claude Desktop) have `mcp.json` files that allow you to configure both **Remote** (HTTP) and **Local** (STDIO) MCP servers.

`mcp.json`:

```
{
    "mcpServers": {
        "local_stdio_server_name": {
            "command": "npx/python/uv/uvx",
            "args": ["some absolute/relative path", "some install
argument"]
        },
        "remote_http_server_name": {
            "url": "mcp_server_url"
        }
    }
}
```

Creating a STDIO Transport Client

Now we'll create an MCP client that uses our `StdioTransport` to communicate with the server. We'll test it the same way we called tools with the in-memory server above.

In [65]:
```
stdio_transport_client = Client(transport_stdio)
print(stdio_transport_client)
```

```
<fastmcp.client.client.Client object at 0x77aa13ad6a80>
```

Now that the client has been defined, we need to call and test it. The process is almost identical to what we've done before. We'll leave it as an exercise to write a function called `test_client` that calls the add tool and prints out the tool list.

▶ Click here for Solution

LangChain Tools

The process for using tools in `STDIO` is similar to HTTP - you just replace `streamablehttp_client` with `stdio_client`. Both return `(read, write)` streams that you use to create a ClientSession. After initializing the session, we use `load_mcp_tools` to convert the MCP tools into LangChain-compatible format.

In [67]:
```
# In an async context manager, convert and print the tools
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
# Configure the STDIO server parameters
server_params = StdioServerParameters(
    command="python",
    args=["stdio_server.py"],
)
```

We create the agent with our LLM and the converted tools, then make a query. Everything stays wrapped in async context managers to keep the child process running and the session alive while the agent executes.

In [66]:
```
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        # Initialize the connection
        await session.initialize()

        # Get tools
        tools = await load_mcp_tools(session)

        agent = create_react_agent(
            model=llm,
            tools=tools,
        )

        agent_response = await agent.ainvoke(
            {"messages": "Use the add tool to add 2 and 1 ."}
        )
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[66], line 1
----> 1 async with stdio_client(server_params) as (read, write):
      2     async with ClientSession(read, write) as session:
      3         # Initialize the connection
      4         await session.initialize()

NameError: name 'stdio_client' is not defined
```

we can print out the responce

In [68]:
```
print(agent_response['messages'][-1].content)
```

```
Yes. I used the add tool to compute 2 + 1. The tool returned 3, so the sum i
s 3.
```

Multiple MCP Servers

The LangChain MCP Adapters library has a module that let's you connect multiple MCP servers and load tools from them. Connecting to each server is similar to earlier when we were creating simple clients to manually call tools. Let's use the `MultiServerMCPClient` module to connect to both our STDIO and HTTP MCP servers.

The client transport configuration is similar to as we did above.

In [70]:
```
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent
```

In [71]:
```
client = MultiServerMCPClient(
    {
        "stdio-client": {
            "command": "python",
            "args": ["stdio_server.py"],
            "transport": "stdio"
        },
        "http-client": {
            "url": f"http://127.0.0.1:{PORT}/mcp",
            "transport": "streamable_http"
        }
    }
)
```

Let's list out the tools and check that tools from both servers exist here.

In [72]:
```
tools = await client.get_tools()
[tool.name for tool in tools]
```

```
INFO:     127.0.0.1:55600 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:55604 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:55618 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:55626 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:55628 - "DELETE /mcp HTTP/1.1" 200 OK
```

```
/opt/conda/lib/python3.12/contextlib.py:105: DeprecationWarning: Use `stream
able_http_client` instead.
  self.gen = func(*args, **kwds)
```

Out[72]:
```
['add', 'subtract', 'add', 'subtract']
```

You should see 2 **add** and 2 **subtract** tools, one from each server as we essentially creted two identical servers with different transports.

Create an Agent

Let's wrap everything up by connecting a LangGraph Agent to our MCP Client and prompting the LLM to use our tools. One advantage of MultiServerMCPClient is it's simpler to work with agents because you don't have to create individual ClientSession objects, manage nested async context managers, or manually initialize connections. Just configure your servers once, get all the tools with a single call, and build your agent - everything else is handled automatically.

- Set our LLM to `gpt-5-nano`
- Pass in the LLM and tools from `MultiServerMCPClient`
- Invoke the agent with a prompt that clearly tells it to use the tools
  - Otherwise the LLM may answer the question based on its foundational knowledge

In [73]:
```
llm = "openai:gpt-5-nano"
agent = create_react_agent(
    model=llm,
    tools=tools
)
agent_response = await agent.ainvoke({"messages": "whats 8 + 7? use tools"})
```

```
INFO:     127.0.0.1:59700 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:59714 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:59720 - "GET /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:59736 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:59740 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:59752 - "DELETE /mcp HTTP/1.1" 200 OK
```

```
/opt/conda/lib/python3.12/contextlib.py:105: DeprecationWarning: Use `stream
able_http_client` instead.
  self.gen = func(*args, **kwds)
```

Let's print out the conversation below to see our MCP tools in action.

In [74]:
```
for i in agent_response['messages']:
    if isinstance(i, HumanMessage):
```

```
        message_type = "HUMAN"
    elif isinstance(i, AIMessage):
        message_type = "AI"
    elif isinstance(i, ToolMessage):
        message_type = "TOOL"
    else:
        message_type = "OTHER"

    if i.content == '':
        i.content = "tool call"

    print(f"[{message_type}] {i.content}")
```

```
[HUMAN] whats 8 + 7? use tools
[AI] tool call
[TOOL] 15
[AI] The result is 15.
```

1. You should see our Human prompt message first ("whats 8 + 7? use tools").
2. Then a AI calls the tool. [AI]
3. The tool response. [TOOL]
4. And finally, the AI response. [HUMAN]

Conclusion

In this lab, we explored the fundamentals of MCP servers and their role in bridging LLMs with external tools and services. You learned how to:

- Connect MCP clients using both **STDIO** and **HTTP** transports
- Interact with tools in a unified and consistent way
- Apply asynchronous patterns for non-blocking communication

MCP servers simplify integration and open the door to scalable, flexible, and powerful LLM-driven applications. With these concepts, you are now equipped to start experimenting, extending, and building your own tool-enabled AI workflows. 🚀

Authors

Joshua Zhou | Data Scientist @ IBM
Joseph Santarcangelo | Data Scientist @ IBM

Copyright © IBM Corporation. All rights reserved.

In [ ]:

```
```